

Atividade 1 – Mini E-commerce Distribuído

Relatório Técnico

Aluno: Anderson Gabriel
Disciplina: FCCPD
Tecnologia: Kotlin + Ktor + SQLite + Docker
Repositório: github.com/andgabx/mini-ecommerce

1. Como a comunicação entre os microserviços foi implementada?

A comunicação entre todos os serviços foi implementada via **HTTP/REST**, usando o **Ktor Client** (engine CIO) como cliente HTTP assíncrono em todos os serviços que fazem chamadas internas.

O **API Gateway** (porta 8080) atua como único ponto de entrada: recebe as requisições do cliente externo, valida o JWT e encaminha ao serviço correto. As comunicações internas são:

- **Gateway → todos os serviços:** proxy reverso com injeção dos headers **X-User-Id**, **X-User-Email** e **X-User-Role** extraídos do JWT validado;
- **products-primary → products-replica:** propagação síncrona de escritas via **POST /internal/replicate**;
- **orders-service → products-service:** consulta de preço e estoque ao criar pedido via **GET /products/{id}**.

Em ambiente Docker, toda a comunicação interna utiliza **HTTPS** com certificado self-signed compartilhado (**keystore.jks**), configurado via **sslConnector** no Ktor Netty engine. O keystore contém tanto a chave privada (**KeyEntry**) quanto o certificado confiável (**TrustedCertEntry**), permitindo que os clientes HTTP verifiquem a identidade do servidor.

2. Qual estratégia de consistência foi adotada na replicação? Por quê?

Foi adotada **consistência forte** (*synchronous replication*).

Funcionamento: ao receber **POST /products**, o **products-primary** propaga o dado para a réplica via **POST /internal/replicate**. Somente após receber confirmação (**200 OK**) da réplica é que o primary confirma a escrita com **201 Created** ao cliente. Se a réplica estiver indisponível, a escrita retorna **503 Service Unavailable** — nenhuma escrita é confirmada de forma parcial.

Justificativa: em um e-commerce, inconsistência de dados de produto (preço, estoque) impacta diretamente pedidos e pode gerar prejuízos financeiros. A consistência forte garante que qualquer réplica que responda a uma leitura já possui o dado mais recente, eliminando leituras sujas. O custo — maior latência nas escritas e falha total se a réplica estiver indisponível — é aceitável dado o escopo.

Propriedade emergente: como escritas falham enquanto a réplica está indisponível,

não há divergência de dados entre os nós. Quando a réplica volta online, ela já está sincronizada com o primary — nenhuma reconciliação adicional é necessária.

Limitação conhecida: existe uma janela de inconsistência se o primary gravar na réplica mas falhar antes de gravar localmente. Essa é uma limitação inerente à abordagem sem log de transações distribuído (WAL compartilhado ou protocolo 2PC).

3. O que acontece com o sistema se o Serviço de Pedidos cair?

O sistema continua operando com **degradação isolada**:

- O Gateway detecta a falha via heartbeat após 2 verificações consecutivas sem resposta (≈ 10 segundos), registra a falha em log com timestamp (`Instant.now()`) e marca o serviço como indisponível;
- Requisições para `/orders/*` retornam imediatamente `503 Service Unavailable`;
- `users-service` e `products-service` continuam operando normalmente — login, cadastro e consulta de produtos não são afetados;
- Quando o serviço volta, o Gateway detecta na próxima verificação de heartbeat, registra `RECOVERED` em log e retoma o roteamento. Como o `orders-service` usa SQLite com persistência em volume Docker, os dados pré-falha são integralmente preservados. Pedidos rejeitados durante a indisponibilidade (`503`) não geraram escrita — não há dados a recuperar.

Demonstração: a seção “Verificar heartbeat” do `README.md` documenta o procedimento de teste: `docker-compose stop orders-service`, aguardar 10s, verificar o `503` via `curl` e o log `RECOVERED` após `docker-compose start orders-service`.

4. Como o JWT garante que um usuário comum não consiga criar produtos?

O JWT gerado no login contém o claim `role` com valor `"user"` ou `"admin"`, além de `userId`, `email` e `exp`.

Fluxo de proteção:

1. No login, o `users-service` embute o `role` no payload do JWT e assina com **HMAC-256** usando a `JWT_SECRET`;
2. Ao receber `POST /products`, o Gateway valida a assinatura do token — qualquer adulteração no payload invalida a assinatura;
3. Se `role` $\neq$ `"admin"`, o Gateway retorna `403 Forbidden` imediatamente, sem encaminhar a requisição ao `products-service`;
4. Senhas são armazenadas com **BCrypt** (salt automático, cost factor padrão) — nenhuma senha em texto plano no banco.

Os serviços internos recebem os claims via headers `X-User-Id`, `X-User-Email` e `X-User-Role` injetados pelo Gateway, que é o único responsável pela validação JWT. Os serviços internos confiam nos headers já validados.

5. Quais limitações a implementação possui em relação a um sistema real?

Ponto único de falha no Gateway

O API Gateway não é replicado. Se cair, nenhuma requisição chega aos serviços. Em produção, seria colocado atrás de um load balancer com múltiplas instâncias.

SQLite e concorrência

SQLite usa lock em nível de arquivo, limitando escritas simultâneas. Em produção, substituído por PostgreSQL ou outro banco orientado a servidor.

Sem transações distribuídas / Saga Pattern

Ao criar um pedido, o estoque do produto não é decrementado. Em produção, seria necessário um padrão Saga com compensação ou uma fila de mensagens (Kafka, RabbitMQ) para coordenar os serviços.

Replicação sem retry automático

Se a réplica estiver indisponível, todas as escritas de produtos falham. Em produção, haveria filas de replicação com retry e dead-letter queue.

Sem refresh token

O JWT expira (24h) e o usuário precisa autenticar novamente. Um sistema real implementaria refresh tokens com rotação.

Certificado self-signed

O TLS usa certificado gerado localmente. Em produção, seria emitido por uma CA confiável (ex: Let's Encrypt via ACME).

Sem rate limiting

O Gateway não limita requisições por cliente, tornando o sistema suscetível a ataques de negação de serviço (DoS) básicos.